

Introducción

El objetivo fue medir el rendimiento de la versión **secuencial (SEQ)** versus la versión **paralela con OpenMP (OMP)** del algoritmo de Lloyd, usando **speedup = T_{seq} / T_{omp}** y CSVs con tiempos por iteración. En PC1 (escritorio) OMP tiene overhead para N pequeños y supera a SEQ a partir de $N=6000$; en PC2 (laptop) OMP es más rápido desde N bajos. Con $N=50000$ se observaron **speedups** de **$\sim 11.5\times$** (PC1) y **$\sim 14.0\times$** (PC2). Estos resultados confirman escalamiento con el tamaño del problema y dependencia del hardware.

Enlaces

- [Repositorio GitHub](#)

Antecedentes

Documentación: Voronoi Stippling

Descripción general

Stippling es una técnica de representación gráfica donde una imagen o figura se aproxima usando **puntos distribuidos en el plano**. El objetivo es que la **densidad y posición de los puntos** transmitan la estructura, contraste y detalles de la imagen.

Para lograr esto de manera algorítmica, se combina:

- **Diagramas de Voronoi** → dividen el plano en celdas según cercanía a cada punto semilla.
- **Algoritmo de Lloyd** → ajusta iterativamente la posición de los puntos hacia el **centroide o centro de masa** de su celda Voronoi.
- **Pesos derivados de la imagen** → las áreas oscuras atraen más puntos, generando mayor densidad (Weighted Voronoi Stippling).

Parámetros principales del algoritmo

1. **N**: número de puntos iniciales (parámetro clave del proyecto).
2. **Imagen de entrada (opcional)**: define la densidad de puntos por oscuridad/brillo. Si no hay imagen, se puede trabajar con distribuciones uniformes o funciones matemáticas.
3. **Iteraciones (I)**: número de repeticiones de Lloyd's Algorithm hasta converger.

4. Distancia usada:

- Euclidiana (clásico).
- Chebyshev o Manhattan (variantes).

5. Pesos/funciones:

- **Weighted**: ponderación según intensidad ($1 - I(x,y)$).
- **Multiplicatively weighted**: cada punto tiene un factor de escala $\rightarrow$ permite simular diferentes tamaños de puntos.
- **Anisotrópico**: distancias escaladas por un ángulo θ y razón α , para estimamiento direccional.

Funcionamiento (flujo básico)

1. Inicializar **N puntos aleatorios** en el área de la imagen/canvas.
2. **Construir el diagrama de Voronoi** con esos puntos.
3. Para cada celda Voronoi:
 - Calcular el **centroide geométrico** (si uniforme).
 - Calcular el **centro de masa ponderado** (si depende de imagen).
4. Mover cada punto al centroide/centro de masa de su celda.
5. Repetir el proceso hasta convergencia o un número fijo de iteraciones.
6. Renderizar los puntos (círculos o dots).

Retornos / Resultados

- **Distribución final de puntos** (coordenadas 2D).
- Imagen stippled que aproxima la forma o gradientes de la imagen de entrada.
- En versión animada (screensaver), se visualiza la evolución iterativa (los puntos “se deslizan” hacia posiciones más uniformes/densas según la imagen).

Variantes importantes

- **Lloyd's Algorithm (Centroidal Voronoi Tessellation)**: puntos uniformemente distribuidos.
- **Weighted Voronoi Stippling**: más puntos en áreas oscuras.
- **Multiplicatively Weighted Voronoi**: permite puntos de distinto tamaño.
- **Anisotropy**: genera patrones direccionales (líneas o texturas).

Reto y propósito del algoritmo

El reto de **Voronoi Stippling** es convertir imágenes o áreas en representaciones basadas únicamente en puntos, distribuidos de manera **eficiente y visualmente coherente**.

Esto implica resolver problemas de:

- **Distribución espacial:** mantener puntos equidistantes evitando agrupamientos innecesarios.
- **Convergencia rápida:** el algoritmo puede ser costoso con miles de puntos.
- **Visualización estética:** los puntos deben transmitir la información visual de manera clara.

En el contexto de tu proyecto de **paralela con OpenMP**, el reto es:

- Computar las celdas Voronoi o aproximaciones para cada iteración.
- Recalcular centroides/centros de masa de manera eficiente.
- Paralelizar el procesamiento de celdas/puntos para acelerar la convergencia.

Cuerpo

Algorítmico Matemático: Voronoi Stippling

1. Diagramas de Voronoi (Región de Influencia)

Un diagrama de Voronoi divide el plano en regiones C_i , cada una asociada a un punto $p_i = (x_i, y_i)$, donde

El criterio de distancia d puede ser:

- Euclidiana: $\sqrt{(x - x_i)^2 + (y - y_i)^2}$
- Chebyshev: $\max(|x - x_i|, |y - y_i|)$
- Manhattan: $|x - x_i| + |y - y_i|$ ([Extra Polynymous](#))

2. Algoritmo de Lloyd (Centroidal Voronoi Tessellation)

Para uniformizar la distribución de puntos se aplica Lloyd iterativamente:

1. Construir el diagrama de Voronoi.
2. Calcular el centroide geométrico de cada región:
3. Reubicar p_i en $(\bar{x}_i, \bar{y}_i)$. Se repite hasta convergencia. ([Extra Polynymous](#))

3. Weighted Voronoi Stippling (por imagen)

En lugar del centroide geométrico, se calcula el centro de masa ponderado según la imagen:

con $I(x, y)$ como brillo o intensidad. ([Extra Polynymous](#))

4. Multiplicatively Weighted Voronoi (tamaños variables)

Para dotar de distintos tamaños (radios r_i) a los puntos, el diagrama considera:

Así se logra una separación proporcional al tamaño. ([Extra Polynymous](#))

5. Anisotropía (distorsión direccional)

Introducción de anisotropía con razón α y ángulo θ :

Esto genera regiones alargadas en dirección preferida. ([Extra Polynymous](#))

6. Implementación simple iterativa

Definir campos sobre la imagen (radio $R(x, y)$, ángulo $\theta(x, y)$, anisotropía $\alpha(x, y)$), luego:

1. Posicionar aleatoriamente N puntos.
2. En cada iteración:
 - Obtener R_i, θ_i, α_i según posición.
 - Construir Voronoi ponderado/anisotrópico.
 - Calcular centro de masa (o centroide).
 - Mover p_i al nuevo punto.
3. Repetir hasta convergencia o límite de iteraciones. ([Extra Polynymous](#), [Extra Polynymous](#))

7. Ajuste de conservación de densidad

Para mantener la “oscuridad total” igual a la densidad visual, se escala uniformemente todo con factor ρ , asegurando:

Así la densidad de puntos representa la luminosidad de la imagen. ([Extra Polynymous](#))

8. Versión escalonada (“Better implementation”)

1. Partir con campos iniciales $R_0(x, y), \theta_0(x, y), \alpha_0(x, y)$ y una imagen I_0 de área A_0 .

2. Establecer un número objetivo de puntos N_f .

3. Iterativamente:

- Generar N_2 temporales con escalamiento proporcional a imagen/área.
- Calcular Voronoi y centro de masa, determinar oscuridad D_i y radio medio $\bar{R}_i$.
- Usar factor:

$$c = \frac{\sum_{i=1}^{N_2} D_i}{\sum_{i=1}^{N_2} \pi \bar{R}_i^2}$$

y asignar nuevos puntos por región:

$$N_i = \text{round}\left(\frac{D_i}{c \pi \bar{R}_i^2}\right), \quad N = \sum_{i=1}^{N_2} N_i$$

- Redistribuir aleatoriamente los N_i dentro de cada región.
- Aplicar iteraciones de Lloyd primero dentro de subregiones y luego en todo el campo.

4. Repetir hasta alcanzar $N \geq N_f$. Esta aproximación gradual mejora la eficiencia y la convergencia. ([Extra Polynymous](#))

Resumen matemático clave

Caso	Distancia modificada	Actualización de puntos
Lloyd clásico	Euclidiana estándar	Centroide
Weighted stippling	Euclidiana + ponderación de imagen	Centro de masa
Multiplicative weights	Euclidiana escalada por $s = 1/r_i$	Incluido en cálculo Voronoi
Anisotropic	Distorsión por α, θ	Incluido en cálculo Voronoi

Diagramas

Diagrama de secuencia

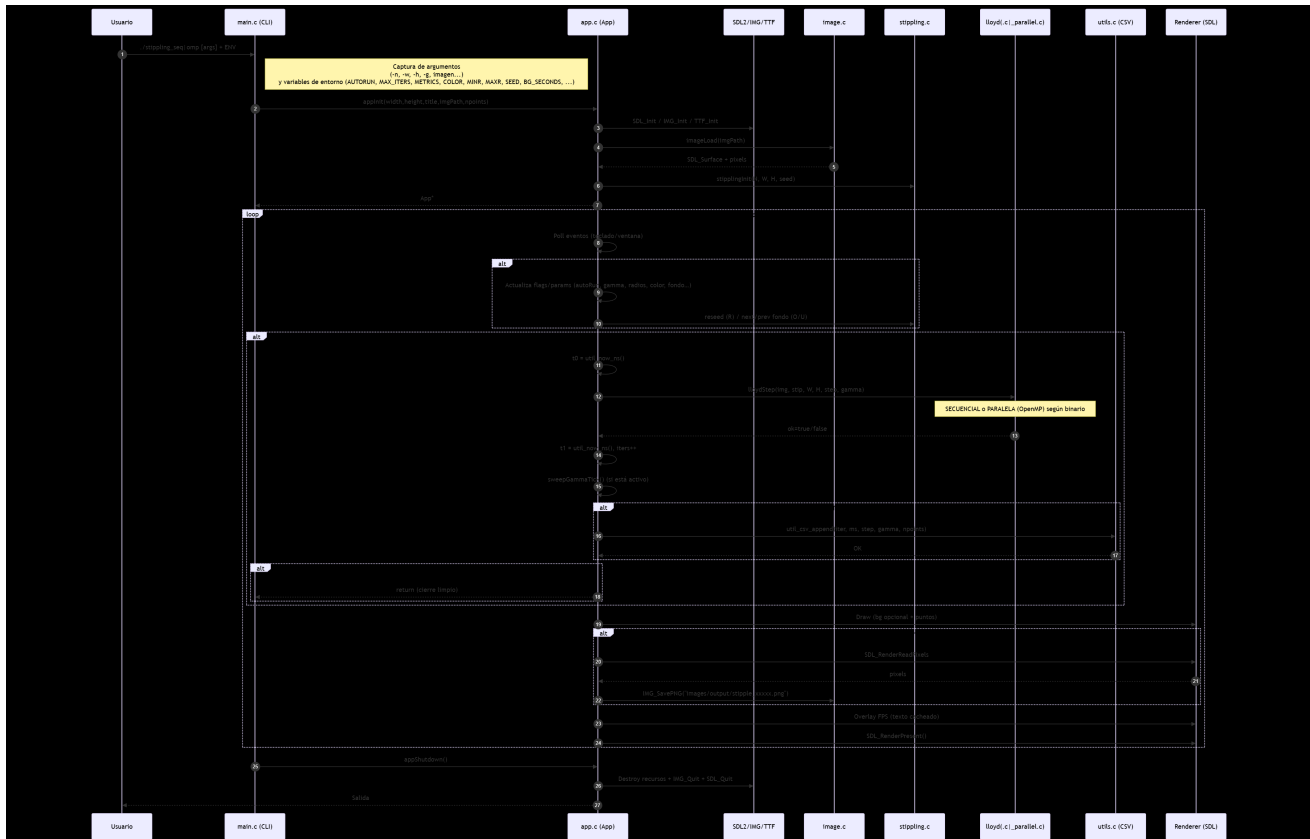
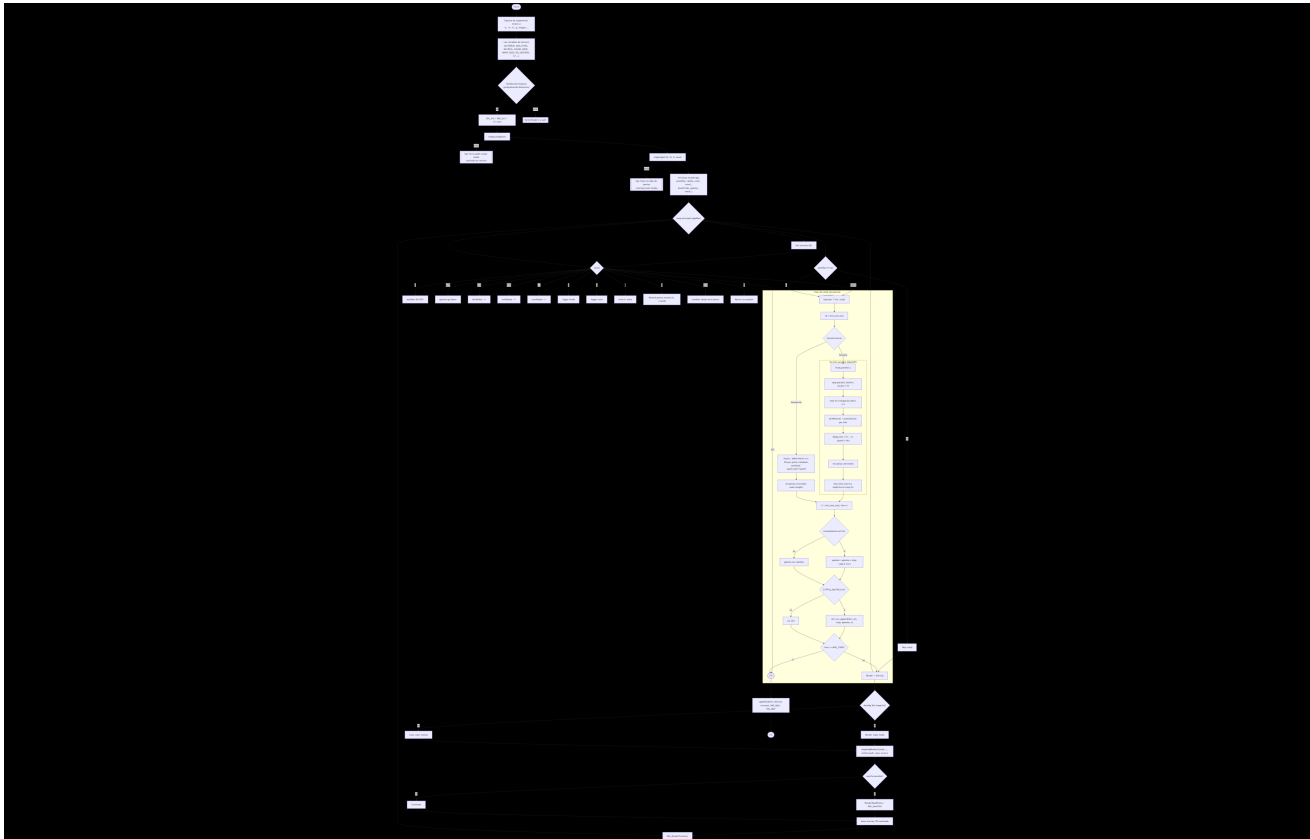


Diagrama de flujo



Bitácora de pruebas

El benchmark utilizado considera las dos partes de los archivos compilados:

- **SEQ** → versión secuencial.
- **OMP** → versión paralela con OpenMP.

El objetivo es medir cuánto mejora (o empeora) el rendimiento al paralelizar; para las pruebas se calculó el **speedup**, esto se define como:

$$\text{Speedup} = \frac{T_{seq}}{T_{par}}$$

donde:

- T_{seq} = tiempo promedio de la versión secuencial.
- T_{par} = tiempo promedio de la versión paralela.
- **Speedup = 1.0** → No hay ganancia, ambas versiones tardan lo mismo.
- **Speedup > 1.0** → La versión paralela es más rápida (hay mejora).
 - Ej: Speedup = 2.0 → OMP es el doble de rápido que SEQ.

– Ej: Speedup = 5.0 → OMP es 5 veces más rápido.

- **Speedup < 1.0** → La versión paralela es más lenta (hay overhead de paralelización).

Benchmark bucle

PC1 - Escritorio

N	Promedio ms (SEQ)	Promedio ms (OMP)	Speedup SEQ/OMP
2000	6.598020	8.775120	0.752
4000	11.783380	12.836300	0.919
6000	17.933300	15.059940	1.194
8000	28.949420	16.253840	1.781
10000	48.921300	22.612800	2.166
12000	68.567910	21.369830	3.209
14000	82.506950	20.197140	3.942
16000	105.261280	23.017500	4.573
18000	123.027270	23.712520	5.188
20000	140.503360	26.834800	5.236

PC2 - Laptop

N	Promedio ms (SEQ)	Promedio ms (OMP)	Speedup SEQ/OMP
2000	11.660620	3.189670	3.659
4000	22.163180	4.908770	4.515
6000	31.974470	5.495560	5.818
8000	51.081030	11.178510	4.571
10000	66.904320	13.873110	4.823
12000	89.123800	20.712340	4.303
14000	119.171460	33.654470	3.542
16000	142.775740	32.417460	4.406
18000	162.633430	27.016560	6.020
20000	174.230730	29.735440	5.856

Benchmark unitario (N = 50000)

PC	N	Promedio ms (SEQ)	Promedio ms (OMP)	Speedup SEQ/OMP
PC escritorio	50000	443.005050	38.483720	11.511
Laptop	50000	823.389900	58.601760	14.051

Evidencia de pruebas

PC1 (Escritorio)

```
0.004s 13:53:10
mnt > d > repositorios > UVG > 2025 > Paralela_P1 (P vs)
>_ ./tests/bench.sh --n-start 2000 --n-end 20000 --n-step 2000 --iters 100 --img images/input/doge.png

Bench N=2000 (iters=100)
→ Ejecutando SEQ (./build/bin/stippling_seq) con N=2000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=2000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N2000_20250905_135323.csv, tests/bench/par/run_N2000_20250905_135323.csv
- Resultados comparación -
|
| Resultados de comparación:
| Promedio ms (SEQ): 6.598920
| Promedio ms (OMP): 8.775120
| Speedup SEQ/OMP : 0.752x
|

Bench N=4000 (iters=100)
→ Ejecutando SEQ (./build/bin/stippling_seq) con N=4000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=4000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N4000_20250905_135328.csv, tests/bench/par/run_N4000_20250905_135328.csv
- Resultados comparación -
|
| Resultados de comparación:
| Promedio ms (SEQ): 11.783830
| Promedio ms (OMP): 12.823630
| Speedup SEQ/OMP : 0.919x
|

Bench N=6000 (iters=100)
→ Ejecutando SEQ (./build/bin/stippling_seq) con N=6000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=6000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N6000_20250905_135335.csv, tests/bench/par/run_N6000_20250905_135335.csv
- Resultados comparación -
|
| Resultados de comparación:
| Promedio ms (SEQ): 17.983300
| Promedio ms (OMP): 15.055940
| Speedup SEQ/OMP : 1.194x
|

Bench N=8000 (iters=100)
→ Ejecutando SEQ (./build/bin/stippling_seq) con N=8000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=8000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N8000_20250905_135343.csv, tests/bench/par/run_N8000_20250905_135343.csv
- Resultados comparación -
|
| Resultados de comparación:
| Promedio ms (SEQ): 28.949420
| Promedio ms (OMP): 16.253840
| Speedup SEQ/OMP : 1.781x
|

Bench N=10000 (iters=100)
→ Ejecutando SEQ (./build/bin/stippling_seq) con N=10000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=10000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N10000_20250905_135355.csv, tests/bench/par/run_N10000_20250905_135355.csv
- Resultados comparación -
|
| Resultados de comparación:
| Promedio ms (SEQ): 48.982130
| Promedio ms (OMP): 22.611190
| Speedup SEQ/OMP : 2.166x
|
```

Bench N=12000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=12000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=12000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N12000_20250905_135409.csv, tests/bench/par/run_N12000_20250905_135409.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 68.567910
Promedio ms (OMP): 21.369830
Speedup SEQ/OMP : 3.209x

Bench N=14000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=14000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=14000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N14000_20250905_135427.csv, tests/bench/par/run_N14000_20250905_135427.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 87.506950
Promedio ms (OMP): 22.197140
Speedup SEQ/OMP : 3.942x

Bench N=16000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=16000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=16000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N16000_20250905_135448.csv, tests/bench/par/run_N16000_20250905_135448.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 105.261280
Promedio ms (OMP): 23.017500
Speedup SEQ/OMP : 4.573x

Bench N=18000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=18000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=18000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N18000_20250905_135511.csv, tests/bench/par/run_N18000_20250905_135511.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 123.027270
Promedio ms (OMP): 23.712520
Speedup SEQ/OMP : 5.188x

Bench N=20000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=20000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=20000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N20000_20250905_135538.csv, tests/bench/par/run_N20000_20250905_135538.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 140.503360
Promedio ms (OMP): 26.834880
Speedup SEQ/OMP : 5.236x

✓OK. Resultados en tests/bench

```

mnt > d > repositorios > UVG > 2025 > Paralela_P1 (p vs)
> ./tests/bench_once.sh --n 50000 --iters 100 --img images/input/doge.png
-> Ejecutando SEQ con N=50000, iters=100, img=images/input/doge.png
-> Ejecutando OMP con N=50000, iters=100, img=images/input/doge.png
Guardado: tests/bench_once/seq/run_N50000_20250905_140343.csv, tests/bench_once/par/run_N50000_20250905_140343.csv
>>> Comparando resultados...
-----
Resultados de comparación:
Promedio ms (SEQ): 443.005050
Promedio ms (OMP): 38.483720
Speedup SEQ/OMP : 11.511x
-----

```

PC2 (Laptop)

```

mnt > d > UVG GitHub Repositorios > 2025 > Paralela_P1 (p vs)
> ./tests/bench.sh --n-start 2000 --n-end 20000 --n-step 2000 --iters 100 --img images/input/doge.png

```

```

Bench N=2000 (iters=100)
-> Ejecutando SEQ (/build/bin/stippling_seq) con N=2000, iters=100, img=images/input/doge.png
-> Ejecutando PAR (/build/bin/stippling_par) con N=2000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N2000_20250905_140137.csv, tests/bench/par/run_N2000_20250905_140137.csv
- Resultados comparación -
Resultados de comparación:
Promedio ms (SEQ): 11.606020
Promedio ms (OMP): 3.172030
Speedup SEQ/OMP : 3.659x

```

```

Bench N=4000 (iters=100)
-> Ejecutando SEQ (/build/bin/stippling_seq) con N=4000, iters=100, img=images/input/doge.png
-> Ejecutando PAR (/build/bin/stippling_par) con N=4000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N4000_20250905_140148.csv, tests/bench/par/run_N4000_20250905_140148.csv
- Resultados comparación -
Resultados de comparación:
Promedio ms (SEQ): 22.163180
Promedio ms (OMP): 4.908770
Speedup SEQ/OMP : 4.515x

```

```

Bench N=6000 (iters=100)
-> Ejecutando SEQ (/build/bin/stippling_seq) con N=6000, iters=100, img=images/input/doge.png
-> Ejecutando PAR (/build/bin/stippling_par) con N=6000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N6000_20250905_140204.csv, tests/bench/par/run_N6000_20250905_140204.csv
- Resultados comparación -
Resultados de comparación:
Promedio ms (SEQ): 31.974470
Promedio ms (OMP): 5.495560
Speedup SEQ/OMP : 5.818x

```

```

Bench N=8000 (iters=100)
-> Ejecutando SEQ (/build/bin/stippling_seq) con N=8000, iters=100, img=images/input/doge.png
-> Ejecutando PAR (/build/bin/stippling_par) con N=8000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N8000_20250905_140220.csv, tests/bench/par/run_N8000_20250905_140220.csv
- Resultados comparación -
Resultados de comparación:
Promedio ms (SEQ): 51.018130
Promedio ms (OMP): 11.160090
Speedup SEQ/OMP : 4.571x

```

Bench N=10000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=10000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=10000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N10000_20250905_140242.csv, tests/bench/par/run_N10000_20250905_140242.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 66.904320
Promedio ms (OMP): 13.873110
Speedup SEQ/OMP : 4.823x

Bench N=12000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=12000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=12000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N12000_20250905_140309.csv, tests/bench/par/run_N12000_20250905_140309.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 89.123800
Promedio ms (OMP): 20.712340
Speedup SEQ/OMP : 4.303x

Bench N=14000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=14000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=14000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N14000_20250905_140341.csv, tests/bench/par/run_N14000_20250905_140341.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 119.171460
Promedio ms (OMP): 33.645430
Speedup SEQ/OMP : 3.542x

Bench N=16000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=16000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=16000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N16000_20250905_140420.csv, tests/bench/par/run_N16000_20250905_140420.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 142.775740
Promedio ms (OMP): 32.404470
Speedup SEQ/OMP : 4.406x

Bench N=18000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=18000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=18000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N18000_20250905_140503.csv, tests/bench/par/run_N18000_20250905_140503.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 162.633430
Promedio ms (OMP): 27.017650
Speedup SEQ/OMP : 6.020x

Bench N=20000 (iters=100)

→ Ejecutando SEQ (./build/bin/stippling_seq) con N=20000, iters=100, img=images/input/doge.png
→ Ejecutando PAR (./build/bin/stippling_par) con N=20000, iters=100, img=images/input/doge.png
✓ Guardado: tests/bench/seq/run_N20000_20250905_140602.csv, tests/bench/par/run_N20000_20250905_140602.csv
- Resultados comparación -

Resultados de comparación:

Promedio ms (SEQ): 174.230730
Promedio ms (OMP): 29.753440
Speedup SEQ/OMP : 5.856x

✓OK. Resultados en tests/bench

mnt > d > UVG GitHub Repositorios > 2025 > Paralela_P1 (p vs)

>_ ./tests/bench_once.sh --n 50000 --iters 100 --img images/input/doge.png

→ Ejecutando SEQ con N=50000, iters=100, img=images/input/doge.png

→ Ejecutando OMP con N=50000, iters=100, img=images/input/doge.png

Guardado: tests/bench_once/seq/run_N50000_20250905_141425.csv, tests/bench_once/par/run_N50000_20250905_141425.csv

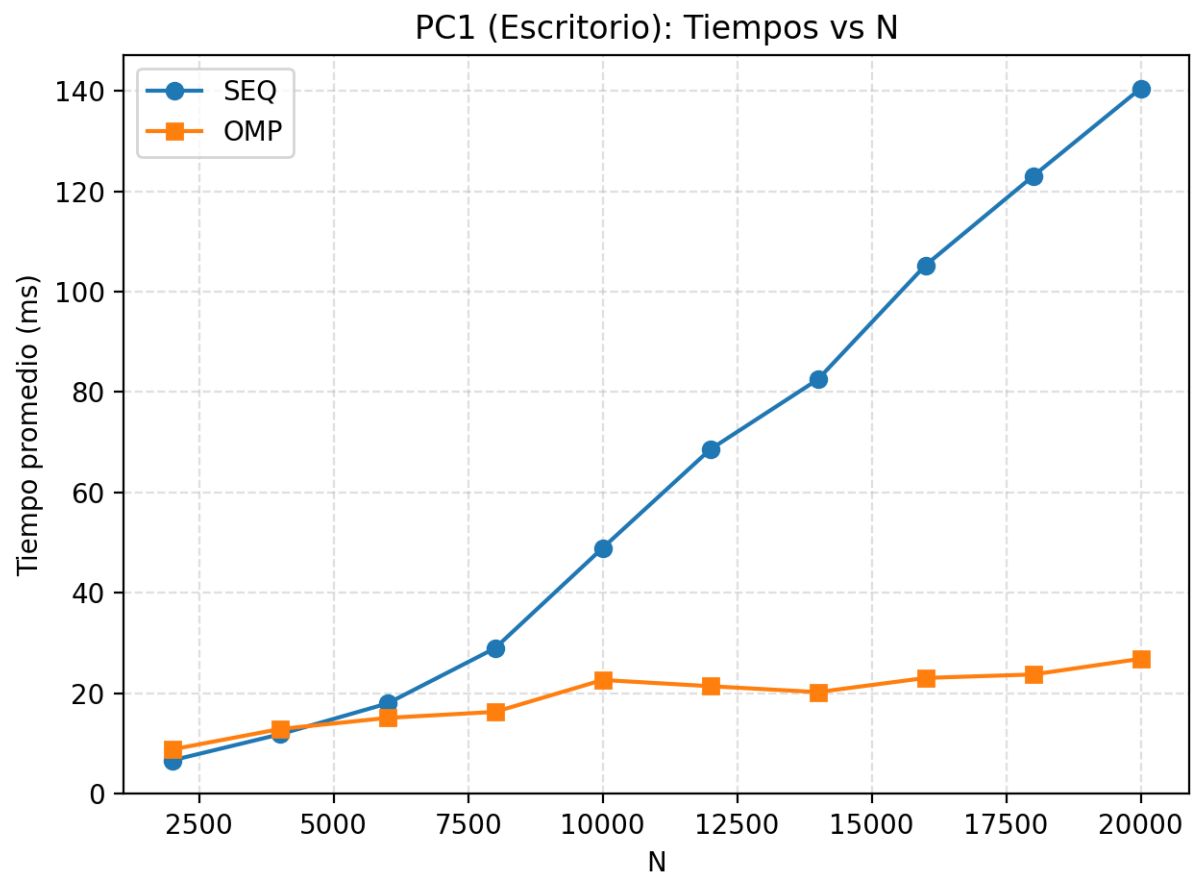
>>> Comparando resultados...

Resultados de comparación:

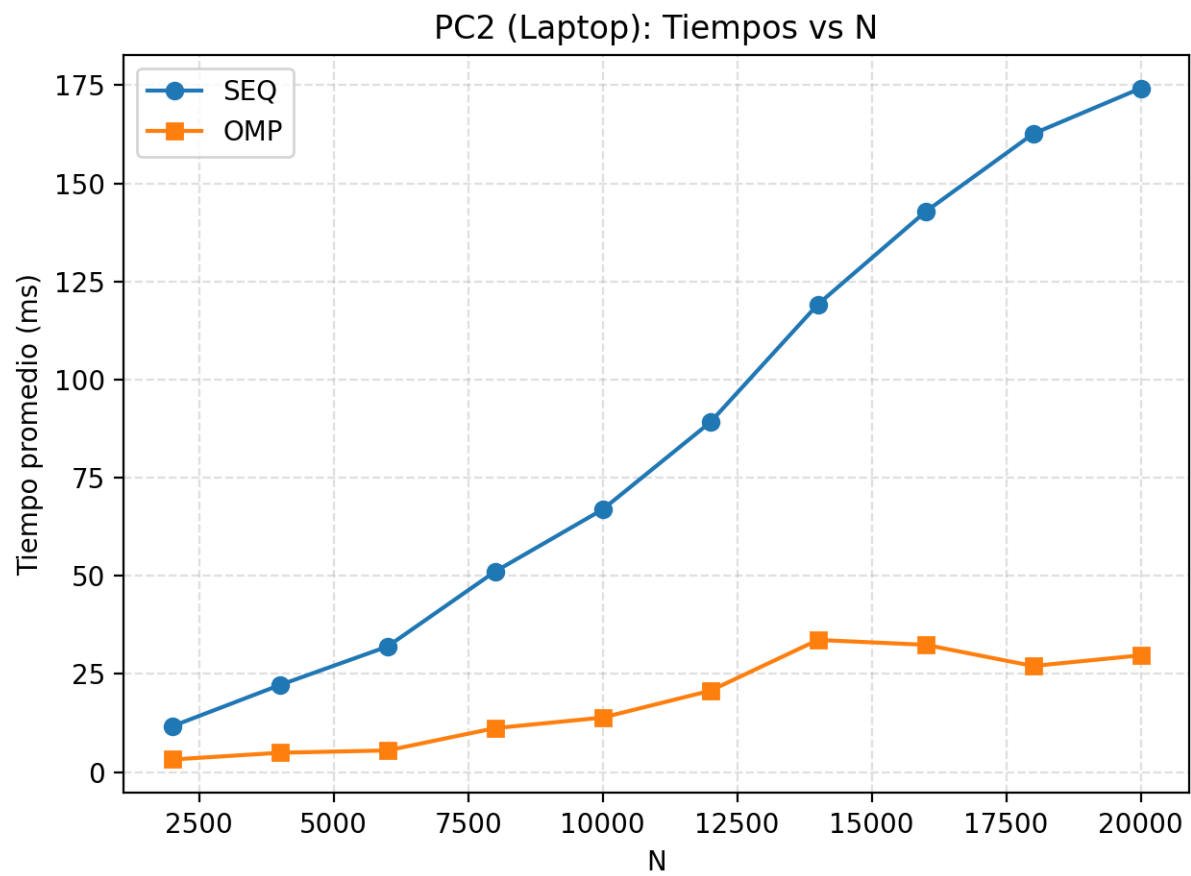
Promedio ms (SEQ): 823.389900
Promedio ms (OMP): 58.601760
Speedup SEQ/OMP : 14.051x

Gráficas

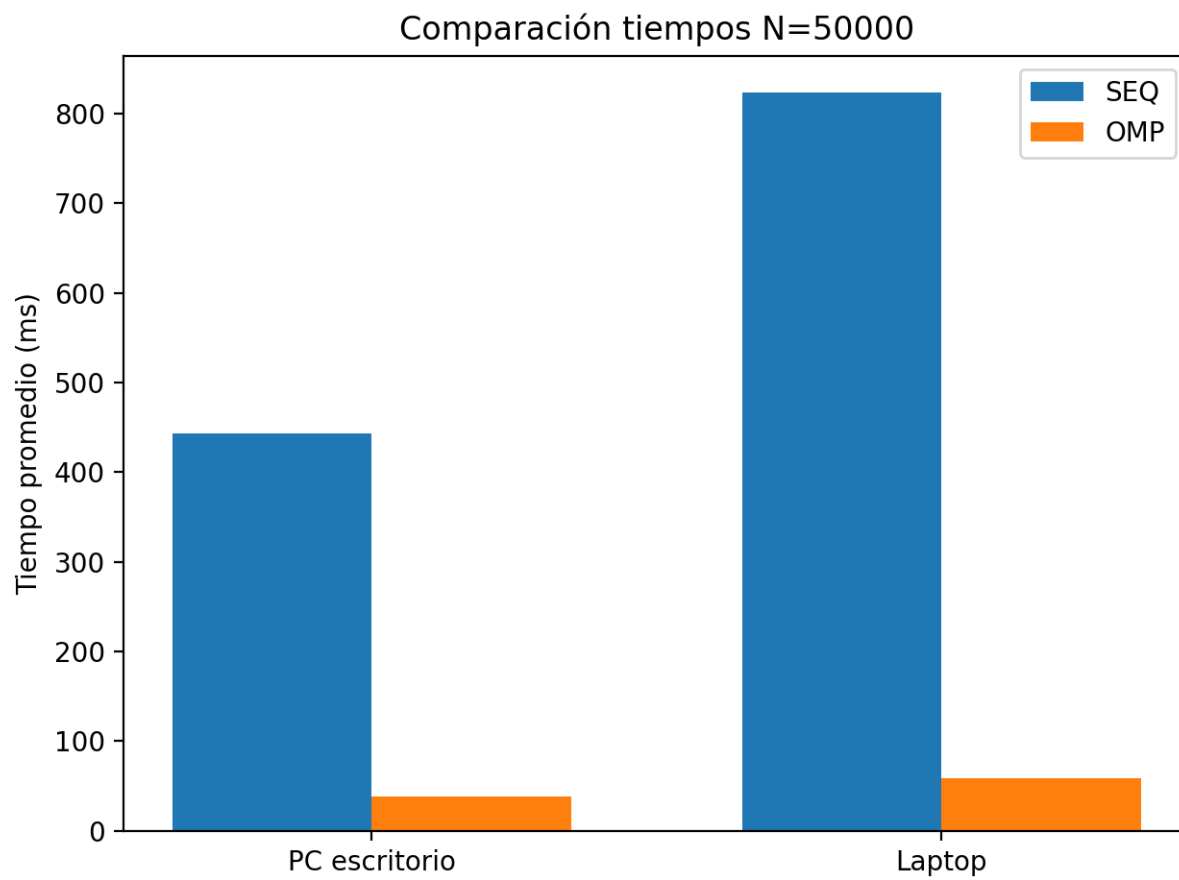
PC1 (Escritorio): Tiempos vs N



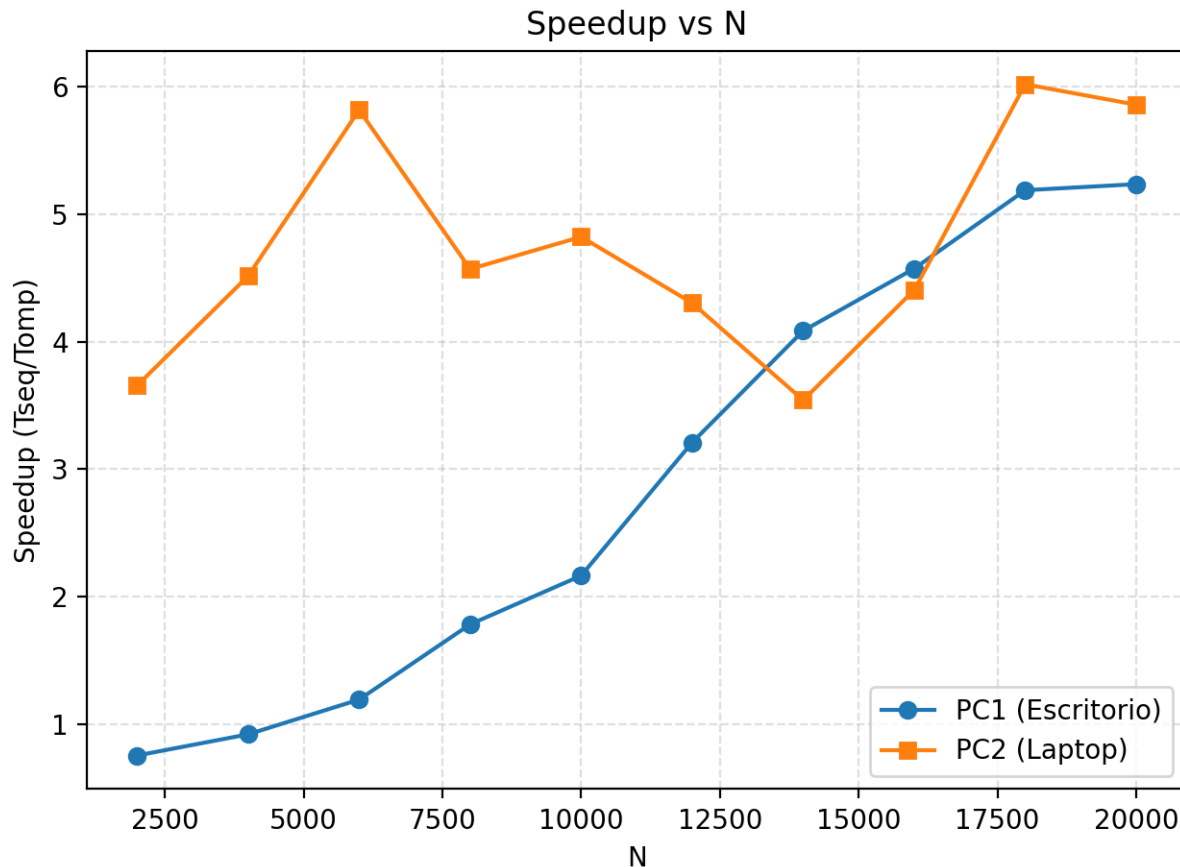
PC2 (Laptop): Tiempos vs N



Comparación tiempos N=50000



Speedup vs N



Interpretación de resultados

- En **problemas pequeños** (N bajos), la versión **OMP** en PC1 llega a ser más lenta que SEQ, debido al overhead de gestión de threads.
- A partir de **N=6000** en PC1, OMP supera claramente a SEQ y escala hasta **5.2x** en N=20000.
- En PC2 (laptop), el **speedup es mayor desde el inicio**, llegando hasta **~6x** en N=18000–20000.
- Para **N=50000**, ambas máquinas muestran una aceleración significativa:
 - PC1 (escritorio): ~11.5x
 - PC2 (laptop): ~14.0x
- Esto refleja que el **algoritmo paralelo escala bien con N**, y que las diferencias de hardware impactan tanto en los tiempos absolutos como en el speedup relativo.

Recomendaciones

- **Tamaño del problema:** usar $N \geq 6000$ en PC1 para ver ganancias; en PC2 el paralelismo rinde incluso con N bajos.
- **Hilos:** fijar OMP_NUM_THREADS al # de núcleos físicos y mantenerlo constante entre corridas comparables.
- **Parámetros fijos para pruebas:** semilla (STIPPLE_SEED), gamma (sin sweep), color ON/OFF, radios, y desactivar rotación de fondos (STIPPLE_BG_SECONDS=0) para reducir ruido en medición.
- **Métricas:** recolectar CSV con STIPPLE_AUTORUN=1 y STIPPLE_MAX_ITERS=K; repetir 3–5 veces y promediar.
- **Warm-up:** descartar las primeras 1–2 iteraciones si hay variabilidad inicial.
- **Afinidad/ruido:** cerrar apps pesadas, usar modo “alto rendimiento” del SO y, si es posible, fijar afinidad (e.g., taskset) para estabilidad.
- **Explorar escalado:** barrer N y graficar ms vs N y speedup vs N para identificar la zona eficiente.

Conclusiones

- La paralelización **no trivial** (buffers por hilo + reducción) elimina contención y habilita **aceleraciones sostenidas** en tamaños grandes.
- Existe un **punto de cruce:** con N pequeño, el overhead de OMP puede superar el beneficio; con N medio/alto, OMP **supera claramente** a SEQ.
- El **hardware** influye: la laptop probada obtiene speedups altos desde N bajos, mientras que el escritorio necesita N mayores para despegar.
- Con $N=50000$ se alcanzan speedups **>10×**, validando que la estrategia paralela es efectiva y escalable para cargas intensivas.

Referencias

- [Esteban Hufstedler: Modified Voronoi Diagrams and Stippling](#) (conceptos de Lloyd, Weighted, Anisotropy)
- [Mike Bostock – ObservableHQ: Voronoi Stippling](#) (visualización interactiva paso a paso).
- [The Coding Train \(Challenge #181 – Image Stippling\)](#): explicación pedagógica y animada.
- [Repositorio de referencia en JS](#)